Conducting Euler-Lagrange Particle Coupled Fluid Dynamics Simulations on OpenFOAM

Nolan Lau Cze Kai **Tengku Amran Sharif** **Goh Teik Beng**
Zhou Yuxiang

1. Introduction

Our simulations involved simulating inkjet printing which involved ejected ink particles moving within air. This is a multiphase situation due to the liquid and gas phase involved. Our approach leveraged on the assumption that as the inks are water-based, the capillary length in air would be approximately 2.7 mm (Rapp, 2017), and thus much larger than the droplet diameters which are of μm size. This allowed us to assume that the ink droplets behaved as rigid spheres in air, adopting a Lagrangian approach for the ink droplets. We conducted our simulations in OpenFOAM (Weller et al., 1998). Due to our specific requirements, a proprietary solver coined as `kinematicDualParcelFoam` was programmed and used and is publicly accessible on GitHub (Lau, 2026). The GitHub also includes an example case for reference. This document will be primarily based on the use of the aforementioned solver. Use of the solver expects experience and understanding in computational fluid dynamics (CFD) and is far beyond the scope of someone who has only followed a guided tutorial before.

2. Hardware and Software Prerequisites

The computationally demanding nature of computational fluid dynamics (CFD) simulations necessitates suitable computational hardware other than for small validation cases. As a minimum recommendation, local execution should employ:

1. a modern multi-core CPU (preferably ≥ 8 physical cores),
2. at least 16–32 GB RAM,
3. and a solid-state drive (SSD).

The simulations are predominantly CPU-bound rather than GPU-bound, as OpenFOAM does not natively exploit GPU acceleration in standard configurations. Furthermore, OpenFOAM works only in Linux environments.

The recommended execution environments are therefore:

1. Windows Subsystem for Linux (WSL2) on Windows,
2. UCL Kathleen HPC Cluster,
3. UCL Myriad HPC Cluster.

The UCL high performance computing (HPC) clusters are especially recommended for bulk work particularly UCL Kathleen HPC Cluster and UCL Myriad HPC Cluster. Students can apply for access with the sponsorship of a permanent staff.

2.1. OpenFOAM Environment

The simulations are conducted using OpenFOAM, an open-source finite volume CFD framework written primarily for Linux-based operating systems. OpenFOAM is not natively supported under standard Windows environments and therefore requires either:

1. a Linux operating system (including through a virtual machine),
2. Windows Subsystem for Linux (WSL2),
3. or execution through a remote Linux HPC cluster (e.g., UCL Myriad).

The recommended OpenFOAM version for this project is OpenFOAM® v2112 as the custom solver was developed for this release. We have found OpenFOAM® v2312 to work as well. Compatibility with older versions is poor but newer versions are likely to work. If installing on a private system, the installation guide should be consulted. UCL Myriad and Kathleen already carry several versions of OpenFOAM.

The OpenFOAM tutorial cases distributed with the software as well as additional documentation online are highly recommended for familiarisation with:

1. theoretical basis of finite volume method (FVM)
2. mesh generation,
3. solver execution,
4. multiphase flow,
5. and lagrangian particle tracking.

2.1.1. Local Installation Using WSL2

For users operating on Windows systems, the recommended approach is to install Ubuntu Linux through Windows Subsystem for Linux 2 (WSL2). This provides a near-native Linux environment while retaining standard Windows usability. Microsoft provides installation instructions.

Ubuntu 22.04 LTS is recommended for compatibility with OpenFOAM-v2212. General Linux familiarity, including use of the shell, bash scripting, SSH and file permissions is assumed for effective use of OpenFOAM and the HPC clusters.

You should be highly familiar with the use of OpenFOAM before proceeding further.

3. Solver `kinematicDualParcelFoam`

The custom solver `kinematicDualParcelFoam` was adapted from the standard OpenFOAM solver `kinematicParcelFoam` and shares close similarities. The main difference is that within the functional class of cloud, the base solver, `kinematicParcelFoam` only has a singular cloud (`kinematicDualParcelFoam`) whereas the custom solver has two (`mainCloud` and `satelliteCloud`). This allows for the simultaneous evolutions of two different types of particles which may have different properties in a single simulation.

3.1. Governing Equations

The computational model adopts an Euler–Lagrange formulation. The carrier phase, namely air, is treated as a continuum and solved on an Eulerian mesh, while droplets are tracked as discrete Lagrangian particles. This approach is well suited to particle-laden flows in which the dispersed phase occupies a small volume fraction but may still exchange momentum with the carrier phase.

The carrier phase is treated as an incompressible Newtonian fluid with the incompressible Navier-Stokes equation solved in the Eulerian frame as

$$\nabla \cdot \mathbf{u} = 0, \tag{1}$$

$$\frac{\mathrm{D}\mathbf{u}}{\mathrm{D}t} = -\frac{1}{\rho}\nabla p + \nu\nabla^2\mathbf{u} + \mathbf{S}, \tag{2}$$

where $\mathbf{u}$ represents fluid velocity, p is pressure, ρ and ν are the fluid density and kinematic viscosity respectively. The source term, $\mathbf{S}$ represents the momentum coupling from the Lagrangian phase, accounting for the impulse transfer from the particles to the fluid. The translational motion of the spherical particles are described by the trajectory equation and Newton's Second Law,

$$\frac{\mathrm{d}\mathbf{x}_\mathcal{P}}{\mathrm{d}t} = \mathbf{u}_\mathcal{P}, \tag{3}$$

$$m_\mathcal{P}\frac{\mathrm{d}\mathbf{u}_\mathcal{P}}{\mathrm{d}t} = \mathbf{F}_d + \mathbf{F}_g, \tag{4}$$

where $\mathbf{x}_\mathcal{P}$ and $\mathbf{u}_\mathcal{P}$ represent the position and velocity of the particle respectively while the particle mass is calculated as

$$m_\mathcal{P} = \rho_\mathcal{P}\frac{\pi D_\mathcal{P}^3}{6}, \tag{5}$$

being a uniform sphere, where $\rho_\mathcal{P}$ represents the particle density and $D_\mathcal{P}$ is the particle diameter. The drag force term $\mathbf{F}_d$ is calculated using the sphere drag correlations by Putnam (1961) as

$$\mathbf{F}_d = C_d\frac{\pi D_\mathcal{P}^2}{8}\rho(\mathbf{u} - \mathbf{u}_\mathcal{P})|\mathbf{u} - \mathbf{u}_\mathcal{P}|, \tag{6}$$

where the drag coefficient, C_d is calculated using the local particle Reynolds number, $Re_\mathcal{P}$ as

$$C_d = \begin{cases} \frac{24}{Re_\mathcal{P}}\left(1 + \frac{1}{6}Re_\mathcal{P}^{2/3}\right) & \text{if} \quad Re_\mathcal{P} \leq 1000 \\ 0.424 & \text{if} \quad Re_\mathcal{P} > 1000 \end{cases}. \tag{7}$$

The gravity force term is calculated as

$$\mathbf{F}_g = m_\mathcal{P}\mathbf{g}\left(1 - \frac{\rho}{\rho_\mathcal{P}}\right), \tag{8}$$

where **g** is the gravitational acceleration vector.

For the present application, additional force contributions such as virtual mass and pressure-gradient effects are small in comparison with drag and gravity and are therefore neglected. However, additional forces may be customised or enabled in the simulation to change (4) as desired. The Euler–Lagrange framework is particularly useful because it permits direct modelling of the force coupling between primary droplets and the carrier flow, while also allowing a large population of secondary droplets to be tracked efficiently.

3.2. Compilation Instructions

The custom solver can be compiled and installed to your version of OpenFOAM using the instructions in the GitHub. However, the OpenFOAM base files are not modifiable in the UCL HPC. Instead, the solver will have to be compiled within your `scratch` and invoked through its path on `scratch`.

4. Running Simulations

This section describes the general workflow for preparing and executing simulations using the custom `kinematicDualParcelFoam` solver. The workflow consists broadly of:

1. mesh generation,
2. configuration of the Eulerian flow solver,
3. configuration of the Lagrangian particle clouds,
4. parallel decomposition,
5. solver execution,
6. and post-processing.

In the OpenFOAM idiom, a simulation is enclosed within a case which is a folder consisting of settings and where results will be stored upon execution. An example case using the solver is provided within the GitHub.

4.1. Case Structure

OpenFOAM cases generally follow the directory structure:

```
case/
   0/
   constant/
   system/
```

The purpose of each directory is:

1. `0/`: Initial and boundary conditions,
2. `constant/`: Mesh, material properties, and cloud settings,
3. `system/`: Numerical schemes and solver controls.

4.2. Mesh Generation

The computational mesh is generated prior to simulation execution. The example case within the GitHub contains a functioning mesh. When creating a mesh for your specific purposes, to ensure highly reproducible and iterative workflows, we suggest the use of parametric mesh generation scripts written in MATLAB (or other programming language).

4.2.1. Parametric `blockMeshDict` Generation

For simple block-structured geometries, OpenFOAM employs the `blockMesh` utility. Its configuration is natively defined in a dictionary within `system/blockMeshDict`. For small changes from the mesh provided in the example case, the corresponding `blockMeshDict` may be manually modified. For more extensive studies, we recommend the use of a MATLAB script to parametrically write the `blockMeshDict` dictionary file so that the geometric properties can be easily changed. Once the MATLAB script writes the `blockMeshDict` to the `system/` directory, the mesh is generated natively in OpenFOAM by running:

```
blockMesh
```

4.2.2. Parametric Gmsh Mesh Generation

For more complex geometries (e.g., non-orthogonal or spline boundaries), we recommend the use of Gmsh for the creation of unstructured meshes (Geuzaine & Remacle, 2009). A similar script should be used to write a dictionary '.geo' file. We then execute Gmsh from the terminal to create a 3D mesh exported in the '.msh' format:

```
gmsh -3 <name>.geo -format msh2
```

Finally, the mesh is converted to OpenFOAM format via the built-in utility:

```
gmshToFoam <name>.msh
```

Regardless of the approach employed (native `blockMesh` or `Gmsh`), the computational mesh quality should subsequently be verified using:

```
checkMesh
```

for low mesh non-orthogonality particularly if complex geometry is involved.

4.3. Initial and Boundary Conditions

The initial and boundary conditions are specified within the `0/` directory. Common files include:

```
0/U
0/p
```

The velocity field file (`U`) specifies velocity boundary conditions and the initial field. It will come to taken on the global velocity field in later time steps. The moving substrate is typically implemented using a Dirichlet boundary condition

```
type            fixedValue;
value           uniform ( 1 0 0 );
```

while inlets and outlets take on

```
type            zeroGradient;
```

and walls take on the no slip condition as

```
 type            noSlip;
```

The pressure field file (p) specifies pressure boundary conditions. Inlets and outlets have their pressures specified as desired. Atmospheric boundaries are specifically specified as zero gauge pressure as

```
type            fixedValue;
value           uniform 0;
```

while walls have

```
type            zeroGradient;
```

4.4. Lagrangian Particle Clouds

The custom solver supports two particle clouds:

1. a coupled primary cloud,
2. and an uncoupled secondary cloud.

The primary cloud represents the larger ballistic droplets responsible for printing and exchanges momentum with the carrier phase. The secondary cloud represents smaller satellite droplets which behave approximately as passive tracers and do not contribute appreciably to the momentum source term.

Cloud properties are generally specified within:

```
constant/
mainCloudProperties
satelliteCloudProperties
```

or equivalent cloud configuration dictionaries.

Typical parameters include:

1. particle diameter,
2. density,
3. injection velocity,
4. parcel count,

5. injection location,
6. drag law,
7. and interaction models.

A simplified example is:

```
solution
{
active          yes;
coupled         yes;
transient       yes;
}

constantProperties
{
rho0            1000;
}

subModels
{
particleForces
{
sphereDrag;
}

injectionModels
{
    model1
    {
        type            patchInjection;
        patchName       nozzlePatch;

        U0              (0 -10 0);
        parcelsPerSecond 100000;
        duration        1;
        flowRateProfile constant 1;
        sizeDistribution
        {
            type fixedValue;
            fixedValueDistribution
            {
                value 22e-6;
            }
        }
    }
}

}
```

Note that the secondary cloud, `satelliteCloud` does not contribute a source term to the momentum equation in (2) so toggling

```
coupled no;
```

is redundant. However, the solver could be modified to have both clouds contributing momentum source terms whereupon they could be separately toggled on and off.

4.5. Numerical Controls

The primary numerical controls are contained within:

```
system/controlDict
system/fvSchemes
system/fvSolution
```

The timestep is constrained using the Courant number:

```
adjustTimeStep yes;
maxCo          0.5;
```

Transient simulations involving particle injection generally require small timesteps owing to high particle velocities and small mesh scales. Furthermore, OpenFOAM only natively supports first-order Euler method time-stepping for particle motion.

4.6. Running Simulations Locally

After mesh generation and case setup, the simulation may be executed serially using:

```
kinematicDualParcelFoam
```

For parallel execution using 8 cores:

```
decomposePar
mpirun -np 8 kinematicDualParcelFoam -parallel
```

After completion:

```
reconstructPar
```

The decomposition settings are specified in:

```
system/decomposeParDict
```

An example configuration for decomposition into 8 subdomain for 8 cores is:

```
numberOfSubdomains 8;

method scotch;
```

4.7. Running Simulations on UCL HPC Clusters

Large simulations should be executed through the UCL HPC batch scheduling system.

Before execution, the appropriate software environment must be loaded:

```
module load beta-modules
module unload gcc-libs
module load gcc-libs/7.3.0
module unload compilers
module load compilers/gnu/7.3.0
module unload mpi
module load mpi/openmpi/3.1.4/gnu-7.3.0
module load openfoamplus/v2112/gnu-7.3.0-64
```

A typical submission script is shown below.

4.7.1. Example HPC Submission Script

```
#!/bin/bash -l

#$ -S /bin/bash
#$ -l h_rt=24:00:0
#$ -l mem=1G
#$ -N test
#$ -pe mpi 7
#$ -wd /myriadfs/home/<user>/Scratch/test

module load beta-modules

module unload gcc-libs
module load gcc-libs/7.3.0

module unload compilers
module load compilers/gnu/7.3.0

module unload mpi
module load mpi/openmpi/3.1.4/gnu-7.3.0

module load openfoamplus/v2112/gnu-7.3.0-64

source $WM_PROJECT_DIR/etc/bashrc

blockMesh

decomposePar -force

gerun \
/myriadfs/home/<userID>/Scratch/kinematicDualParcelFoam/kinematicDualParcelFoam \
-parallel
reconstructPar
```

```
rm -r processor*
```

4.8. Submitting Jobs

The script may be submitted using:

```
qsub submit.sh
```

Job status may be monitored using:

```
qstat
```

A job may be terminated with:

```
qdel <jobID>
```

4.9. Post-Processing

Simulation results may be visualised using ParaView:

```
paraFoam
```

Note that while 3D visualisation is supposedly possible using UCL HPC Clusters with an X server, we have found this unreliable. We recommend doing 3D post-processing with ParaView locally on your personal computer.

References

Geuzaine, C., & Remacle, J.-F. (2009). Gmsh: A 3-D finite element mesh generator with built-in pre- and post-processing facilities. *International Journal for Numerical Methods in Engineering*, *79*(11), 1309–1331. https://doi.org/10.1002/nme.2579

Lau, N. C. K. (2026). kinematicDualParcelFoam. Retrieved February 7, 2026, from https://github.com/Nolanlau/KinematicDualParcelCloud/tree/main

Putnam, A. (1961). Integratable form of droplet drag coefficient. *Journal of the American Rocket Society*, *31*(10), 1467–1468.

Rapp, B. E. (2017). Chapter 21 - capillarity. In B. E. Rapp (Ed.), *Microfluidics: Modelling, Mechanics and Mathematics* (pp. 445–451). Elsevier. https://doi.org/10.1016/B978-1-4557-3141-1.50021-6

Weller, H. G., Tabor, G., Jasak, H., & Fureby, C. (1998). A tensorial approach to computational continuum mechanics using object-oriented techniques. *Computer in Physics*, *12*(6), 620–631. https://doi.org/10.1063/1.168744